

1교시: 이미지와 레이어의 이해

수업 목표

- base image, layer, cache, tag, digest의 차이를 설명한다.
- Docker image가 파일 시스템 변경의 누적이라는 관점으로 build output을 읽는다.
- tag와 digest를 운영 신뢰성 관점에서 구분한다.

50분 흐름

시간	활동	비중	학생 산출
0-8분	Day 1 container 실행 복습	설명 15%	image/container 구분
8-20분	layer/base image 설명	설명 25%	layer note
20-32분	cache/tag/digest 비교	설명 20%	비교표
32-44분	docker history nginx:1.27-alpine 읽기	실행 25%	history evidence
44-50분	Dockerfile build 연결	설명 15%	질문 1개

Visual 1: Docker 이미지 레이어 이해

Week 2 Day 2
Lesson 1

Docker 이미지 레이어 이해

이미지는 여러 개의 읽기 전용 레이어의 합으로 구성됩니다.

불변 이미지
이미지 ID(Digest)로 내용을 정확히 식별

1. 이미지 레이어 구조

상위

Layer 3 (애플리케이션 코드) 파일 추가/변경

Layer 2 (설정 파일 추가) 파일 추가/변경

Layer 1 (패키지 설치) 파일 추가/변경

Base Image (OS / 라이브러리) 기본 파일 시스템

하위

각 레이어 = 이전 레이어 위에 적용된 파일 시스템 변경
변경된 내용만 저장하여 용량 절약

2. 빌드 Cache (재사용)

이전 빌드

다음 빌드 (일부 변경)

Step 1 FROM ubuntu:22.04 재사용

Step 2 RUN apt-get update 재사용

Step 3 RUN apt-get install -y curl 재사용

Step 4 COPY ./app 다시 빌드 (변경됨)

Step 5 CMD ["python", "app.py"] 다시 빌드

Cache 히트 = 재사용 → 빌드 시간 단축
변경된 단계부터 다시 빌드

3. Tag (태그)

사용자가 붙이는 이름표 (별명)

- 사람이 기억하기 쉬운 라벨
- 같은 태그가 다른 이미지를 가리킬 수 있음 (변경 가능)

예시: myapp:1.0, myapp:latest

4. Digest (다이제스트)

이미지 내용의 고유 식별자

- 이미지 내용을 해시로 표현
- 동일한 내용이면 항상 동일
- 변경 불가 (불변 식별자)

예시: sha256:3a1b2c4d5e6f7g8h...

핵심 포인트

- ✓ 이미지는 Base Image 위에 여러 Layer가 쌓인 구조
- ✓ Layer는 읽기 전용이며, 변경 사항만 저장
- ✓ 빌드 시 Cache를 재사용하여 불필요한 빌드를 줄임
- ✓ Tag는 사람이 사용하는 이름, Digest는 이미지의 신원

재사용 + 불변성

효율적이고 신뢰 가능한 배포의 핵심

이 이미지는 base image 위에 layer가 쌓이고, 변경되지 않은 build step은 cache로 재사용되며, tag와 digest가 image를 식별하는 방식이 다르다는 점을 보여준다.

핵심 설명

image는 하나의 큰 파일이 아니라 여러 layer의 합으로 이해하면 좋다. 각 layer는 파일 추가, 수정, 삭제 같은 filesystem 변경이다. Dockerfile의 instruction은 새 layer를 만들거나 metadata를 바꿀 수 있다. 이 구조 덕분에 같은 base image나 변경되지 않은 build step을 재사용할 수 있다.

tag는 사람이 붙이는 이름표다. paperclip-static-site:day2처럼 읽기 쉽지만, 같은 tag가 시간이 지나 다른 내용을 가리킬 수 있다. digest는 image content를 기준으로 한 식별자라 더 엄격하다. 운영에서는 tag를 기록하되, 중요한 배포나 감사에서는 digest까지 확인한다.

학술/시스템 관점

image layer는 운영체제의 filesystem 개념과 연결해서 이해한다. 파일은 path 아래에 저장되고, directory tree는 여러 파일과 metadata의 상태를 나타낸다. container image는 이 directory tree의 변경분을 layer로 쌓아 표현한다. 그래서 COPY index.html styles.css ./는 단순 복사 명령이 아니라 image filesystem에 새로운 변경분을 남기는 build step이다.

Docker storage 문서는 container가 기본적으로 read-only image layer 위에 writable container layer를 붙여 실행된다고 설명한다. 이 말은 build 결과와 runtime 변경이 분리된다는 뜻이다. image를 다시 만들지 않고 container 안에서 파일을 수정하면 그 변경은 image가 아니라 해당 container의 writable layer에 남는다. container를 삭제하면 그 변경도 사라질 수 있다.

OCI image specification은 image를 manifest, config, layer 같은 content-addressed artifact로 다룬다. 초급 단계에서 manifest 구조를 외울 필요는 없지만, digest가 "그럴듯한 이름"이 아니라 content identity와 연결된다는 점은 중요하다.

실행: history 확인

```
docker history nginx:1.27-alpine
```

Linux 사전 테스트 핵심 출력

IMAGE	CREATED	CREATED BY	SIZE
<missing>	13 months ago	CMD ["nginx" "-g" "daemon off;"]	0B
<missing>	13 months ago	EXPOSE map[80/tcp:{}]	0B
<missing>	15 months ago	ADD alpine-minirootfs...	7.83MB

history는 image가 어떤 instruction과 base layer 위에 만들어졌는지 보는 도구다. SIZE가 0B인 줄도 의미가 있다. 모든 instruction이 파일을 추가하는 것은 아니기 때문이다.

수치화: layer가 줄이는 시간과 크기

layer와 cache의 장점은 "빠르다" 또는 "효율적이다"로만 설명하면 학생에게 잘 남지 않는다. 1교시에 서는 Day 2 실습 image를 기준으로 실제 숫자를 같이 본다.

측정 명령:

```
cd week2/day2/labs/static-site
/usr/bin/time -f 'elapsed=%e sec' docker build -t paperclip-static-site:metrics .
/usr/bin/time -f 'elapsed=%e sec' docker build --no-cache -t paperclip-static-site:metrics-nocache .
docker history paperclip-static-site:metrics
docker images paperclip-static-site
```

Linux 사전 테스트 측정값:

항목	측정값	해석
cache 적용 재빌드	1.54초	WORKDIR, COPY step이 CACHED로 재사용됨
--no-cache 비교 빌드	약 1.50초	이 예제는 너무 작아 시간 차이가 거의 없음
전체 image 크기	48.2MB	nginx:1.27-alpine 기반 image 전체 크기
실습 소스 COPY layer	2.34kB	index.html, styles.css가 만든 변경분
소스 layer 비율	약 0.0049%	2.34kB / 48.2MB 기준

이 숫자의 핵심은 시간보다 크기다. 이 실습처럼 base image가 이미 있고 HTML/CSS만 바뀌는 경우, 변경된 layer는 전체 image 48.2MB 중 약 2.34kB뿐이다. 비율로는 약 0.0049%다. 즉 layer 구조가 없었다면 학생은 "작은 HTML 수정"도 전체 실행 환경과 하나의 큰 덩어리로 다시 다뤄야 한다. layer 구조가 있으면 변경되지 않은 base image와 이전 build step은 재사용하고, 바뀐 소스 layer만 새로 만들 수 있다.

단, 위 시간값은 PC 성능, Docker Desktop/WSL2 상태, base image pull 여부, BuildKit cache 상태에 따라 달라진다. 그래서 수업에서는 절대 시간이 아니라 다음 세 가지를 evidence로 본다.

evidence	학생이 확인할 질문
build output의 CACHED	어떤 step이 재사용되었는가
docker history의 SIZE	어떤 instruction이 image size를 늘렸는가
전체 image 크기 대비 변경 layer 크기	이번 수정이 전체 artifact에서 몇 %인가

판단 표

용어	뜻	운영 질문
Base image	시작점이 되는 image	출처와 version을 신뢰할 수 있는가
Layer	filesystem 변경 단위	어떤 instruction이 용량을 늘렸는가
Cache	변경 없는 step 재사용	예상과 다르게 예전 결과가 재사용됐는가
Tag	사람이 읽는 이름표	latest만 쓰고 있지는 않은가
Digest	content 기반 식별자	같은 image를 정확히 재현할 수 있는가

운영 사고 예시

상황	잘못된 해석	올바른 해석
container 안 파일을 수정했다	image가 수정되었다	container writable layer가 수정되었을 가능성이 크다
같은 tag로 다시 pull했다	항상 같은 image다	tag는 이동할 수 있으므로 digest/image ID를 확인한다
build가 너무 빠르다	아무 일도 안 했다	cache가 instruction 결과를 재사용했을 수 있다
image가 예상보다 크다	Docker가 원래 무겁다	어떤 layer가 커졌는지 docker history로 확인한다

핵심 유의사항

image와 container는 초급 단계에서 가장 자주 섞이는 개념이다. 이 교시에서는 용어 자체보다 상태 구분이 중요하다. image는 실행 전 artifact이고, container는 image를 실행한 instance다. 같은 image에서 container를 여러 개 만들 수 있고, container 하나를 지워도 image는 남는다.

layer도 "폴더가 여러 개 쌓인다" 정도로만 이해하면 부족하다. 핵심은 변경분이라는 점이다. COPY가 파일을 넣으면 그 instruction 결과가 layer로 남고, 나중에 같은 instruction과 같은 입력이 유지되면 cache가 재사용될 수 있다.

tag는 사람 친화적 이름이고 digest는 content 친화적 식별자다. latest라는 tag는 "가장 안정적"이라는 뜻이 아니다. 단지 누군가가 그렇게 붙인 tag일 뿐이다. 운영에서 같은 이름을 다시 pull했는데 내용이 달라지는 사고는 tag를 digest처럼 믿을 때 생긴다.

docker history의 <missing>은 초급자에게 불안해 보일 수 있다. 이것은 image가 깨졌다는 뜻이 아니다. 중간 layer의 image ID가 표시되지 않는 경우이며, history를 읽을 때는 CREATED BY와 SIZE를 중심으로 보면 된다.

자주 놓치는 포인트

놓치는 지점	올바른 기준
image를 실행 중인 프로그램으로 생각함	실행 중인 것은 container이고 image는 실행 재료다
container 안 수정이 image 수정이라고 생각함	runtime writable layer와 build image layer를 분리한다
latest를 최신 안정판으로 생각함	latest는 움직일 수 있는 tag일 뿐이다
size가 0B인 instruction을 무의미하다고 봄	metadata나 실행 기본값도 image 동작에 영향을 준다
cache hit를 오류로 봄	입력이 같으면 이전 build 결과를 재사용하는 정상 동작이다

학생 기록 템플릿

```
## Lesson 1 Layer Evidence
- 확인한 image:
- 가장 큰 layer:
- size가 0B인 instruction:
- 내가 추가한 source layer:
- 전체 image size:
- source layer 비율:
- tag와 digest/image ID 차이에 대한 한 문장:
```

50분 진행 흐름

0~10분에는 Day 1에서 실행했던 nginx container를 다시 떠올린다. "어제 실행한 것은 image였나 container였나"를 먼저 확인하면 image/container 혼동을 빠르게 드러낼 수 있다.

10~25분에는 레이어 그림을 보면서 filesystem 변경분이라는 표현을 반복한다. PC 부품 비유를 쓴다면 base image는 기본 장착된 OS 디스크 이미지, layer는 설치/복사로 생긴 변경 이력, cache는 이미 만들어 둔 부품 조립 결과 재사용으로 연결한다.

25~40분에는 docker history를 실제로 읽는다. SIZE가 있는 줄과 0B인 줄을 각각 하나씩 고르고, 왜 size가 다른지 설명한다.

40~50분에는 다음 교시 Dockerfile로 연결한다. "Dockerfile의 어떤 줄이 새로운 layer를 만들까"라는 질문을 남기면 2교시의 instruction 학습이 자연스럽게 이어진다.

평가 기준

기준	2점 evidence
layer 이해	image를 layer 누적으로 설명했다.
cache 이해	cache hit와 rebuild 필요 상황을 구분했다.
tag/digest	tag와 digest 차이를 설명했다.
실행 evidence	docker history 출력에서 하나 이상의 instruction을 찾았다.

공식 근거 링크

- Docker Docs: What is an image?, <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/>
- Docker Docs: docker image history, <https://docs.docker.com/reference/cli/docker/image/history/>
- Docker Docs: Storage, <https://docs.docker.com/engine/storage/>
- OCI Image Specification, <https://specs.opencontainers.org/image-spec/>